Teaching the Machine to Teach Game Design

A Case Study in AI-Assisted Curriculum Development

VIDEO GAME CREATION

SLHE Game Programming - Video Game Creation

Why This Document Exists

This course - a 30-meeting, two-semester homeschool class teaching game programming across eight engines and three coding styles - was built with heavy, sustained use of an AI assistant (Claude) as a production partner: drafting lesson content, catching bugs in example code, generating diagrams, and maintaining a set of written production rules that grew directly out of real classroom testing. That process is itself worth documenting - not as a sales pitch for AI, but as an honest record of what the collaboration actually looked like: where it worked, where it needed a human's hands-on verification, and how a shared set of standards emerged from real mistakes rather than being designed up front.

A note on how this document was assembled: the idea for a case study came up mid-project, after a large amount of the actual back-and-forth conversation had already happened and aged out of what could be recovered verbatim. This first version is reconstructed from the project's own paper trail - the Lesson Production Standards document, the running Book Notes & To-Do scratchpad, and the project task list - rather than from a raw conversation transcript. It is being kept going, written for an outside reader, from this point forward. See the open item at the end of this document about exploring the account-level data export as a way to recover more of the earlier, now-unavailable conversation detail and enrich this account further.

The Shape of the Collaboration

Robert is the subject-matter expert and the final authority throughout: a working game developer and homeschool teacher who already knew what he wanted students to learn, which engines mattered, and - critically - who tested nearly every piece of example code himself, in the actual engines, before it reached students. The AI's role was closer to a very fast, very broad junior collaborator: drafting the first version of a lesson across eight different engines at once, remembering and cross-checking consistency rules a human would have to flip through dozens of documents to verify, and converting a verbal classroom observation into a written, versioned production rule the same day it was raised.

The pattern that shows up again and again in the project's own notes is this: Robert tests something hands-on, finds it doesn't actually work as drafted (or works but inconsistently with another engine), and that single concrete finding becomes a new or revised standing rule applied to everything built afterward. The AI did not originate the course's pedagogy or its engine choices; it drafted at scale against rules that a human kept sharpening through direct testing.

Why Eight Engines, One Lesson

The course's defining structural choice - one shared weekly lesson, built out independently in Scratch, Godot, Unity, GameMaker, GDevelop, TIC-80, 001 Game Creator, and Unreal, spanning text code, drag-and-drop blocks, and pure visual node graphs - is unusual for a game-programming course, where most pick a single engine. It lets a student choose whichever style matches how they think, without missing any concept the class covers, and it let Robert accommodate students already learning in different tools without running separate classes.

That choice is also what made the AI genuinely useful as a production partner rather than just a drafting tool: maintaining eight parallel, conceptually-identical implementations by hand, every week, for thirty meetings, is exactly the kind of bookkeeping-heavy consistency work that benefits from a collaborator that can hold all eight at once and flag where they've quietly diverged.

A Rule Written After a Mistake: Case Studies

Several of the course's Production Standards exist because something concrete went wrong first, then got named and prevented from recurring. Two are worth walking through in detail, because they illustrate different failure shapes.

The Unreal Drift (Rule 5: Engine Deviations Require a Driving Reason)

Unreal's lesson content had quietly drifted into using flat-color Materials where every other engine used the actual supplied sprite artwork - close enough to function, but not actually consistent with the rest of the course. Nothing was factually wrong; it simply wasn't the same experience a student would get in any other engine, and nobody had asked why until the drift was noticed during a review. That prompted Standards Rule 5: any time one engine's content deviates from the others, the deviation needs an explicit, stated reason, or it gets fixed to match. It's an honest, relatable shape for any multi-platform project: a rule written after the mistake it exists to prevent, not before.

The Opposite Shape: A Rule Written After an Accidental Correct Answer (Rule 13)

Not every rule came from catching an error. Reviewing an early lesson surfaced an asymmetry that had been sitting unnoticed for weeks: Unity's code already routed shared game state (the enemy counter, the win flag) through a small dedicated GameManager object, while every other engine had that same state scattered across whichever object happened to need it first - a spawner here, a collision handler there. Unity hadn't been designed that way on purpose; it had simply been written that way first, and nobody had asked 'why does only one engine do this' until someone did. The answer was obvious once asked: a dedicated object whose only job is holding shared state is simply the better habit. That became Standards Rule 13, and the other seven engines were retrofitted to match - a rule discovered by noticing an accidental correct answer nobody had generalized yet, rather than a mistake.

The GameMaker Visual Targeting Problem (and Its Own Correction)

A smaller, more technical example shows the value of hands-on verification specifically. An early draft of GameMaker's drag-and-drop (Visual) instructions for tinting a newly-spawned enemy called for dragging the new instance's reference into a generic targeting block - which, on direct testing, turned out not to actually support that use case at all. Robert found a workaround using a different generic block and typed field, which was shipped as the fix and written up as a named exception to the course's own 'prefer purpose-built blocks' rule.

Later, continued hands-on testing in GameMaker turned up a cleaner answer: a purpose-built scoping block (Instance - Apply To) could do the whole job correctly, if nested the right way in the block canvas - meaning the earlier 'exception' wasn't actually a limit of the tool, it was a limit of how far the first pass of testing had looked. Both the lesson content and the standing rule were corrected again once that was found. The lesson drawn from this, recorded directly in the project's own notes: a purpose-built block's targeting options looking limited doesn't necessarily mean the block can't do the job - check whether a scope-setting block changes what's available before concluding a workaround is the only option. That is a lesson about tool investigation generally, independent of AI.

What the AI Was Good At

- Holding eight parallel implementations in mind at once and flagging where they'd quietly diverged in naming, structure, or missing state (Rules 11, 12, 13).
- Converting a single verbal classroom observation ("a student read WinLabel as Window Label") into an immediately actionable, project-wide audit and a named standing rule the same day.
- Drafting at full scale, quickly - a complete lesson's worth of handout, teacher's key, slide deck, and diagrams across four or more engines in one working session - so that review time could go toward correctness rather than first-draft production.
- Keeping a long, growing, and sometimes self-contradicting set of production rules internally consistent, and flagging when new content conflicted with an existing rule before it shipped.

Where Human Hands-On Testing Was Non-Negotiable

The project's own notes are candid about this, and it's the most important finding for anyone considering a similar collaboration: AI-drafted code that looks correct and follows every stated rule can still be wrong in an engine-specific way that only running it reveals. Three examples from this project, in increasing order of subtlety:

- A spawn-loop spacing bug (enemies overlapping and mutually destroying each other at spawn, invisible unless you actually pressed play and looked) existed quietly across multiple engines until Robert found it by testing Godot himself, then an audit of the remaining engines found it was present - or safely absent for identifiable architectural reasons - everywhere else.
- GameMaker's drag-and-drop path twice needed a hands-on correction to targeting logic that looked plausible on paper (see above) and was wrong, then wrong in a subtler way, before a third pass actually got it right.

- A 'complete method' code listing format, meant to show students a whole updated method rather than just the new lines (a standing production rule, Rule 14), silently dropped previously-added optional sound-effect code when it was regenerated for a later lesson - a drift invisible in the document itself and only caught because a teacher, not the AI, noticed students' games had gone quiet.

None of these were caught by the AI reviewing its own output. They were caught by a human running the actual software.

The Stalemate: A Deliberately Unfixed Bug

Not every bug found during development was treated as something to eliminate. An early lesson's win/lose logic has a real gap: enemies destroyed by the player reduce the 'enemies remaining' counter, but enemies that escape only reduce the player's lives - never the counter. A round where some enemies are destroyed and some escape can end with no win, no loss, and the restart key doing nothing at all.

Rather than quietly fixing this, the decision was made to leave it in on purpose, as a diagnose-and-fix exercise students encounter directly - and, crucially, to let it recur in a later lesson (where it becomes more conspicuous, since a stuck round can now happen at any of several game states, not just one) so that it becomes irritating enough that a student is motivated to actually go back and fix their own project, rather than having the fix handed to them. The bug was given a name for the manuscript - the Stalemate - specifically to make it a shared, nameable concept across the course and the eventual book, the same way a well-known software bug class gets a name once enough people have run into it. It stands as a useful, concrete example of a production decision that required a human judgment call: an AI collaborator surfaced the bug and could have 'fixed' it unprompted; the better pedagogical move was to recognize it as a feature of the lesson plan and preserve it.

A Standing Practice, Not a One-Time Build

Perhaps the most important structural fact about this collaboration is that it never stopped being a conversation. The course's Lesson Production Standards document - the rulebook governing consistency, naming, diagram requirements, code completeness, and more - grew from one rule to fifteen over the course of building just the first eleven of thirty lessons, almost entirely in response to things a human found by actually teaching and testing, not from an upfront design pass. A parallel running document, kept deliberately informal (add freely, reorganize only in occasional dedicated passes), captured narrative anecdotes, half-formed ideas, and open questions as they came up, so that good material for an eventual book draft wasn't lost to the pace of weekly production.

That rhythm - build, test by hand, find a gap, name it as a rule, apply the rule going forward, periodically review and consolidate - is arguably the real subject of this case study, more than any single piece of AI-generated content. The tool accelerated drafting and consistency-checking considerably; it did not replace the testing loop, and the project's own records make clear that every time it was treated as if it could, something shipped broken.

A Fourth Source Found: the Raw Conversation

The account-level data export (Settings > Privacy > Export in the Claude product) turned out to be a dead end for this purpose - it only captures claude.ai web-chat history, not Cowork desktop

sessions, and this project's conversation wasn't in it at all. The actual raw material was found instead in the Claude desktop app's own local session cache, which Robert located and exported himself after hearing about the claude.ai export's limits. That cache held the complete, un-summarized conversation for this entire project - 391 real exchanges from the very first message through the present day, plus two short separate Claude Dispatch runs (remote sessions Robert kicked off outside the main desktop conversation) that generated and corrected Lesson 6's sound effects.

That raw material has been extracted into a structured, line-by-line archive (Book/Conversation_Archive/) built specifically to be walked in small segments over time, with each exchange getting room for direct commentary: what worked, what went sideways, where the approach visibly changed. That annotated walkthrough is this document's planned companion and eventual source - not yet started, but the hard part (actually finding and recovering the full, real conversation) is now done.